COMP8715 SOFTWARE ENGINEERING PROJECT

PULSE Technical Architecture and Scoring System

Current Expo React Native and Firebase Implementation

Document detail	Value
Product	PULSE balanced life mobile application
Purpose	Submission technical architecture and scoring system documentation
Implementation examined	Expo and React Native client with Firebase backend services
Document version	1.0
Prepared	October 2026

Document Overview

This document explains how PULSE is implemented as a cross platform wellbeing application and how its scoring model converts meaningful user activity into five domain scores and a whole life indicator. It is written for assessors and project contributors who need to understand the current architecture, data boundaries, key runtime decisions, and the limits that should be addressed before release.

Contents

1 Architecture Summary

2 Expo and React Native Client Architecture
3 Firebase Backend Architecture
4 Navigation and Feature Modules
5 Data Architecture and Real Time Flows
6 Scoring System
7 Privacy Security and Release Considerations
8 Testing and Quality Approach
9 Source Code Map
10 Architecture Decisions

Reading Guide

Sections 1 to 5 describe the running application. Section 6 specifies the scoring behaviour in enough detail to verify calculations against the source. Sections 7 and 8 identify implementation constraints and quality practices. Statements about future work are labelled as recommendations rather than presented as completed functionality.

1 Architecture Summary

PULSE is a cross platform wellbeing application built with Expo and React Native. It combines account management, domain based activity logging, personal finance tools, daily missions, progress analytics, social interaction and guided onboarding. The architecture keeps platform interface code, application state, Firebase access and scoring calculations separate so that feature pages can evolve without duplicating core data or score logic.

The mobile client runs on iOS, Android and web from one JavaScript codebase. Firebase Authentication identifies the current account, Cloud Firestore stores structured user and social data, and Cloud Storage stores media. The scoring engine is a pure JavaScript module, which makes the mathematical rules reusable and straightforward to unit test.

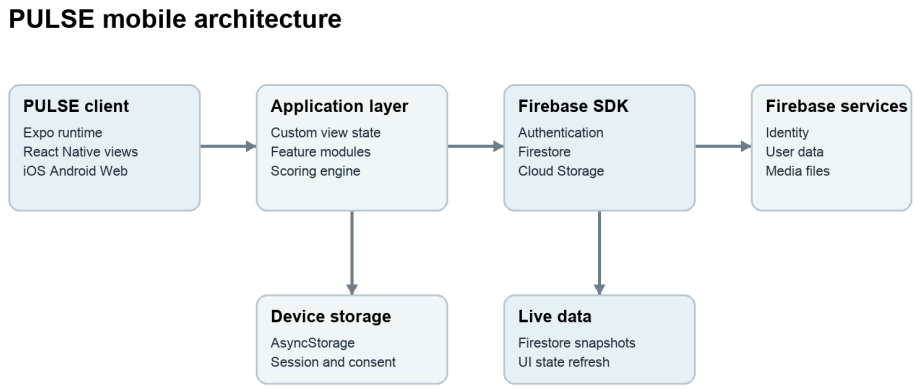


Figure 1. The PULSE client separates presentation, application modules, local device state and Firebase services.

Layer	Responsibility	Current implementation
Presentation	Render accessible mobile screens and capture user actions.	React Native components and Expo runtime.
Application	Coordinate routes, authentication state, onboarding and feature behaviour.	App.js plus page, hook, tutorial and feature modules.
Domain logic	Calculate scores and choose user missions without network dependencies.	scoringEngine.js and missionEngine.js.
Data access	Read and write account, activity, social and finance records.	Firestore service modules and Firebase SDK.
Platform services	Persist session and consent locally; manage cloud identity, data and files.	AsyncStorage, Firebase Authentication, Firestore and Storage.

2 Expo and React Native Client Architecture

2.1 Expo Runtime and Platform Scope

Expo provides the runtime, Metro development workflow and native module integration for a single React Native application. The app configuration declares the pulse URI scheme, portrait orientation, the com.pulse.balancedlife application identifier for both iOS and Android, and Metro

single-page output for web. Expo packages such as expo status bar, expo font, expo image picker and the native date time picker are integrated through the application configuration. This gives the project one JavaScript codebase while retaining device capabilities where needed.

Configuration concern	Current setting	Architecture impact
Application identity	iOS and Android use com.pulse.balancedlife; custom scheme is pulse.	Provides stable native package identity and a base for future deep links.
Screen and assets	Portrait orientation, light interface style and bundled asset patterns.	Keeps the visual runtime predictable and includes app assets in builds.
Native capabilities	Image library and camera permissions are declared through expo image picker.	Supports attaching post images while making permission use explicit to the user.
Web output	Metro bundler with single output mode.	Allows the same application to render on web, subject to platform specific behaviour.
Build service	An EAS project identifier is present in Expo extra configuration.	Connects the application configuration to managed build and distribution workflows.

2.2 Startup Lifecycle

App.js is the composition root. It creates state for the active view, Firebase user, user profile, onboarding progress, selected domain, return tab, score refresh version, tutorial request and consent lifecycle. On launch it waits for fonts, reads the device level privacy notice acknowledgement from AsyncStorage, and subscribes to Firebase authentication changes. The auth listener is the source of truth for the signed-in session; a retry replaces the subscription so Firebase can replay current state.

When Firebase returns a user, the app ensures the user profile exists, loads its current data, reconciles account consent, and routes the user to consent, the initial entry quiz or the authenticated app shell. A boolean ref identifies a newly created account before the auth callback runs, avoiding ambiguity between first registration and a restored session. The asynchronous effects also hold an alive flag so an unmounted screen does not receive a late state update. Returning users benefit from persisted native authentication rather than being returned to the sign in screen after the process is killed.

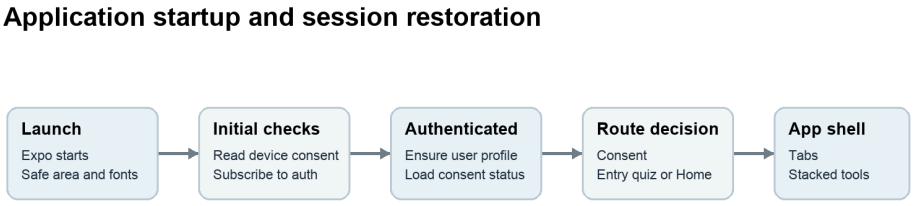


Figure 2. The launch sequence gates authenticated content behind profile, consent and onboarding checks.

2.3 Navigation Model

PULSE currently uses a small custom view state machine instead of React Navigation. The root state identifies splash, authentication, consent and entry quiz screens, then exposes four authenticated tab views: Home, Features, Social and Profile. Finance, domain activity, to do and daily missions are stacked views launched from a tab and returned to the originating tab. `tabReturn` retains the opening tab, while `activeDomain` identifies which generic domain page should render. Android hardware-back handling maps stacked or authentication views back to their appropriate predecessor rather than exiting unexpectedly. This is a practical approach for the current scope, although adopting a navigation library would make deep links, state restoration and navigation analytics easier as the product grows.

2.4 Module Boundaries

Area	Boundary	Examples
Pages	Feature screens own layout and user interaction, not shared persistence rules.	Home, Finance, Social, Profile, PhysicalActivity.
Components	Reusable visual and interaction building blocks.	AppShell, UI controls, share prompt and tutorial components.
Firestore services	Feature specific operations isolate Firebase calls from screens.	scoring, finance, savings, social, messaging and users modules.
Pure logic	Deterministic calculations avoid UI or Firebase imports.	scoringEngine and missionEngine.
Privacy	Consent notice wording and records are maintained separately.	ConsentModal, ConsentScreen and consentStore.

3 Firebase Backend Architecture

3.1 Firebase Initialisation

The Firebase bootstrap module reads EXPO_PUBLIC_FIREBASE variables supplied through the Expo build environment: API key, auth domain, project identifier, storage bucket, messaging sender identifier and app identifier. These values identify the Firebase project and are bundled into the client; they are not a server secret boundary. Protection therefore depends on Firebase Authentication and correctly deployed Firestore and Storage rules. A development warning is emitted when the API key is missing, helping a contributor detect an absent local environment file without exposing a credential in the source repository.

The module creates one Firebase app instance and uses getApps before initialiseApp to prevent duplicate application errors during Metro Fast Refresh. Equivalent guards wrap Authentication and Firestore initialisation: native auth falls back to getAuth if it was already created, and Firestore falls back to getFirestore after an attempted second initialisation. The module exports the configured clients so all feature services share one configuration and no screen embeds Firebase project details.

3.2 Session Persistence

Authentication uses getReactNativePersistence with AsyncStorage on iOS and Android. This is important because native React Native environments do not provide browser IndexedDB or localStorage. On web the standard getAuth path is used. The implementation falls back to getAuth when the instance was already initialised, which protects the developer experience during Fast Refresh while retaining a persistent returning-user session.

3.3 Firestore Transport and Cache Decision

Firestore is initialised with forced long polling and a memory-only local cache. The project selected this configuration to avoid WebSocket and persistent-cache assertion failures observed on constrained or unstable networks. The trade off is deliberate: connection recovery is more predictable, but offline persistence is not available and a cold launch must refetch data. The document therefore treats online availability as a current operational assumption, not an offline-first guarantee.

3.4 Backend Services

Firebase service	Role in PULSE	Representative data or action

Authentication	Creates and restores Email Password accounts.	Sign up, log in, log out and password reset.
Cloud Firestore	Stores user profiles, activity records, scores, finance data and social relationships.	Reflections, actions, budgets, posts, friendships and conversations.
Cloud Storage	Stores user supplied post media.	Post image upload and retrieval.
AsyncStorage	Stores device scoped state that should survive app restarts.	Native auth persistence and local privacy notice acknowledgement.

3.5 Request Semantics and Failure Handling

Firestore service methods return small result objects such as `{ ok: true }` or `{ ok: false, error }` rather than allowing raw SDK exceptions to cross directly into page components. Pages can then surface a controlled message while the service retains responsibility for the operation. Writes use server timestamps where ordering matters, preventing device clock differences from becoming the authoritative event time.

Action logging accepts an optional stable event identifier. When present, it uses a merge write to `users/{uid}/actions/{eventId}` rather than an auto-generated document. This makes task-completion scoring idempotent: toggling or retrying the same to do event cannot accumulate duplicate reward points. Social post creation uses an application-level query to limit a normal check-in to one per user per day. The source explicitly recognises that two racing clients could pass this check, so a stronger production guarantee would require a transaction or a server-side enforcement mechanism.

4 Navigation and Feature Modules

The authenticated product is organised around four root destinations. Home presents the five life domains and mission progress. Features offers direct tool access. Social combines posts, friends and direct messages. Profile presents account settings and progress analytics. Feature pages use service modules to persist events, then update the score version state so score dependent views can refresh without placing score calculations inside individual screen components.

Module	User outcome	Technical mechanism
Onboarding	A new user establishes baseline ratings for five life domains.	Entry quiz writes a baseline scoring record.
Domain activity	A user records reflections and meaningful actions.	Firestore collections plus pure score calculations.

Productivity	A user maintains a to do list and task completion evidence.	To do page contributes productivity actions.
Health	A user logs physical activity and health related activity.	Physical activity page maps activity to health evidence.
Finance	A user records budgets, expenses, accounts and savings plans.	Finance and savings Firestore service modules.
Missions	A user receives daily actions selected from lower score domains.	Mission engine selects three daily tasks and preserves history.
Social	A user can publish, manage friends and exchange messages.	Firestore snapshots with deterministic conversation pair identifiers.

5 Data Architecture and Real Time Flows

5.1 User Centred Firestore Model

Most personal data is scoped below the signed in user's document. This design aligns ownership and security rules with a simple path structure. The user document holds profile, consent, onboarding baseline, current domain scores, frozen week-start scores, last reflection scores and whole-life metrics. Nested collections hold domain reflections, actions, weekly snapshots, missions and other feature records. Social data adds shared collections for posts, friendship records and conversations, where the client uses a deterministic pair identifier to ensure the two participants locate the same conversation.

Path pattern	Write model	Reason for the structure
users/{uid}	Profile and derived score summary are updated in place.	Provides the authenticated application with a compact user aggregate.
users/{uid}/reflections/{id}	Append-only daily rating events.	Preserves raw reflection evidence for the current score period.
users/{uid}/actions/{id}	Append or deterministic merge when a stable event ID exists.	Supports auditability and idempotent task completion scoring.
users/{uid}/weeklyScores/{weekId}	One merged snapshot per ISO week.	Retains score breakdowns without duplicating weekly buckets.

posts, friendships and conversations	Shared top-level documents guarded by participant or friendship rules.	Supports social access patterns that cross user ownership boundaries.

5.2 Live Updates

Where a view benefits from immediate changes, Firestore `onSnapshot` listeners stream updates back into React state. This is used for social posts, friendship state and messages because the interface should change as another account interacts. Like and comment counts are derived from live subcollection snapshots, avoiding a separately maintained counter that can drift from the source records. Deterministic conversation IDs based on the participant pair let both participants resolve the same conversation document.

Most individual logging actions follow a request-response pattern: a page records an event, refreshes relevant data, then increments the shared `scoreVersion` state in App.js so score-dependent views can recompute or reload summaries. The distinction is intentional: continuously listening to every personal activity collection would increase reads and lifecycle complexity, whereas social features benefit directly from reactive updates.

5.3 Weekly Score Transaction Flow

Score computation reads the user baseline and derives the current ISO week identifier. At the start of a new Monday-based week it freezes `weekStartScores` from the prior domain score or onboarding baseline and stores `weekStartId`. Subsequent recomputations in the same week use those frozen values, preventing repeated score calculations from repeatedly applying the same award point to an already updated score. It then queries reflection and action subcollections where the server timestamp is on or after the week start, buckets events by domain, derives the R A C breakdown, updates the user aggregate and merges a single weekly snapshot keyed by the week ID.

The weekly snapshot includes each domain's reflection, action, consistency, observed, award and final values. This keeps an audit trail for explaining a historical score. If an event query fails, the method continues using empty current-period event buckets and logs a diagnostic instead of failing the entire dashboard. This is resilient for a student project, but the resulting score must be interpreted carefully because a permission or connectivity problem can produce an incomplete current-week calculation.

5.4 Main Data Categories

Category	Examples	Purpose
Identity and profile	users, privacyConsent, onboarding state	Establish account context and route eligibility.

Wellbeing evidence	reflections, actions, physical activities, to do items	Supply recent score inputs by life domain.
Score history	weekly scores and score breakdowns	Show progress and retain the input breakdown for review.
Financial planning	budgets, expenses, accounts, saving plans and entries	Provide finance tools and finance domain evidence.
Social	posts, friends, conversations and messages	Support interaction while separating each user's personal records.

6 Scoring System

PULSE measures five domains: spirituality, family and friends, work and productivity, health, and financial wellbeing. Each domain starts from an onboarding rating. Weekly evidence then updates the domain through three inputs: self reflection, action completion and consistency. The design deliberately gives actions the highest weight, while still recognising how the user feels and whether activity is sustained across days.

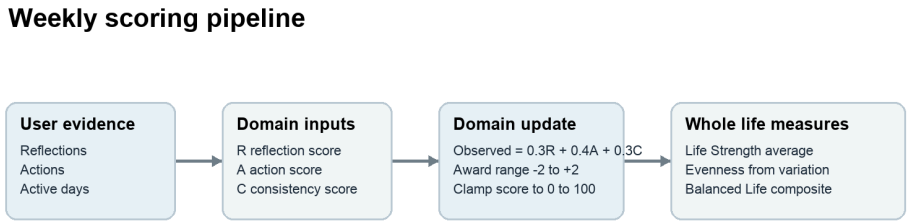


Figure 3. Evidence is normalised into a domain update before whole life measures are calculated.

6.1 Domain Configuration

Domain	Weekly target	Illustrative evidence
Spirituality	200 points	Journal reflection; mindfulness or prayer.
Family and friends	200 points	Meaningful connection log; social mission.

Work and productivity	250 points	Important task completion; focus session.
Health	300 points	Exercise activity; sleep log.
Financial wellbeing	200 points	Expense log; budget review or savings action.

6.2 Domain Calculation

A reflection rating is recorded on a one to five scale and converted to a percentage by multiplying the average rating by 20. The action score is the points earned divided by the configured weekly target, capped at 100. The consistency score is the number of days containing meaningful activity divided by seven, also capped at 100.

Component	Calculation	Weight
Reflection R	Average recent rating multiplied by 20	30 percent
Action A	Minimum of 100 and points earned divided by weekly target multiplied by 100	40 percent
Consistency C	Minimum of 7 and active days divided by 7 multiplied by 100	30 percent
Observed	0.3R plus 0.4A plus 0.3C	Input to award

The observed score is not written directly as the new domain score. It becomes an award point from minus two to plus two using (Observed / 100 x 4) minus 2. That award is added to the prior domain score and the result is clamped between 0 and 100. This gradual update prevents one unusually strong or weak week from abruptly replacing the established baseline.

6.3 No Activity Behaviour

When a domain contains neither actions nor reflections for the current period, the implementation keeps the prior score unchanged and awards zero points. When the user submits reflections but no actions, it also keeps the prior score unchanged. A new score movement therefore requires action evidence, while reflections influence the size and direction of that movement when actions exist.

6.4 Whole Life Measures

Measure	Calculation	Interpretation
Life Strength	Average of the five domain scores.	Overall level across the life domains.

Evenness	100 minus twice the standard deviation of the five scores, clamped to 0 to 100.	Higher when domain scores are closer together.
Balanced Life Score	0.5 x Life Strength plus 0.5 x Evenness.	Combines progress with balance across domains.

7 Privacy Security and Release Considerations

PULSE implements two privacy gates. A device level notice acknowledgement is stored before the sign up form collects identity details. Account level consent is then recorded after registration and before the entry quiz. The consent implementation refers to Australian Privacy Act principles and stores a local record as well as an account record so the user is not unnecessarily prompted after using a different device.

Firebase security rules should remain part of every release review. Existing rules give the owner read and write access to recognised user subcollections such as budgets, expenses, accounts, to do items, physical activities, reflections, actions and weekly scores. Social post and conversation access is constrained through friendship or participant checks. However, the parent `users/{uid}` document is currently readable by every authenticated account, which is broader than strict per-user privacy if profile documents contain sensitive fields. Newly introduced collections such as saving plans, saving entries, finance metadata and tutorial progress also need explicit rule coverage and emulator or device verification. Without matched rules, the default deny behaviour can prevent a correctly coded feature from reading or writing its data.

Release consideration	Why it matters	Recommended action
Privacy contact and policy	The current configuration has placeholder or absent public contact and policy values.	Publish a privacy policy, contact channel and data location statement before public release.
Firestore rule coverage	New collection paths may not be covered by existing security rules.	Test every feature path with least privilege rules before deployment.
Storage access	Authenticated media reads may be broader than a friends only interpretation.	Review intended post visibility and tighten storage rules if required.
Profile document visibility	The users root-document read rule is authenticated but not owner-only.	Minimise sensitive root fields or restrict reads to the owner and expose safe public profiles separately.

Declared Android permissions	Camera and media permissions are used for posts; microphone permission should be justified by an implemented feature.	Audit Android manifest permissions and remove any capability not needed in the release build.
Offline behaviour	The memory cache choice intentionally removes persistent Firestore offline storage.	Communicate the online requirement or plan an offline resilience strategy.
Developer utilities	Development data generators must not be exposed to production users.	Gate or remove debugging utilities from release builds.

8 Testing and Quality Approach

The source tree contains Jest based tests across the current application. The architecture supports focused tests because scoring and mission selection logic have no Firestore or user interface dependency. These should be treated as the primary regression boundary for changes to thresholds, weights and numerical calculations. Component and integration tests should then verify that a user action creates the correct persisted evidence and that screens refresh when the score version changes.

Test level	Focus	Examples
Unit	Pure deterministic logic.	Score formulas, clamping, no activity handling, mission selection.
Service	Firebase data transformations and path ownership.	Create action, load weekly score, deterministic conversation ID.
Component	Screen state and user feedback.	Authentication states, consent gates, form validation, loading behaviour.
Manual device	Expo device behaviour and backend integration.	Session survives relaunch, uploads work, social updates arrive, rules permit intended user only.

9 Source Code Map

The following files are the principal reference points for this document. The list is intentionally concise so a reviewer can trace architecture claims to the current codebase without treating it as a complete file inventory.

Path	Reason to inspect
pulse/src/App.js	Application composition, lifecycle gates, view state and authenticated shell.
pulse/src/firebase.js	Firebase bootstrap, native session persistence, Firestore transport and cache choices.
pulse/src/scoring/scoringEngine.js	Domain definitions, weekly targets, formulae and global score calculations.
pulse/src/firestore/scoring.js	Score baselines, activity evidence and weekly snapshot persistence.
pulse/src/firestore/finance.js and savings.js	Financial records, budgets, savings plans and entries.
pulse/src/missions/missionEngine.js	Daily mission selection and history behaviour.
pulse/firestore.rules and storage.rules	Firebase access policy and media permissions.
pulse/app.json	Expo package identifiers, platform configuration and plugins.

This document represents the current implementation examined in the project repository. Configuration values and backend rule deployments can change independently of the client source, so the release recommendations in Section 7 should be rechecked immediately before a public deployment.

10 Architecture Decisions

The implementation reflects several deliberate choices that are appropriate for the present project scope. These choices also make the boundaries for future improvement visible to the team.

Decision	Rationale	Implication
One Expo codebase	Delivers iOS, Android and web from a shared React Native implementation.	Reduces duplication while retaining access to native modules.
Persistent native authentication	AsyncStorage supplies an explicit persistence layer for React Native.	A returning user keeps their session after a mobile app restart.
Long polling and memory cache	Avoids unstable WebSocket and persistent cache failure modes seen during development.	Prioritises connection reliability over persistent offline access.

Pure scoring engine	Keeps numerical policy separate from views and Firestore calls.	Formula changes can be unit tested without rendering a screen or mocking a backend.
Custom view state	Keeps navigation dependencies small for the current screen set.	A navigation library is the natural next step if deep links or more complex flows are required.